



Høgskulen
på Vestlandet

Forstudie

ELNIKK-07, Selvlagd 8-Bit cpu med FPGA-lagd skjermkort og hovedkort

Sondre Johnsen

Jacob Kvasnes

10. februar 2026

Dokumentkontroll

Rapportens tittel: Forprosjektrapport for ELNIKK-07, Selvlagd 8-Bit cpu med FPGA-lagd skjermkort og hovedkort	Dato/Versjon
	Rapportnummer: B021E-XX
Forfatter(e): Sondre Johnsen Jacob Kvasnes	Studieretning: Elektronikk
	Antall sider m/vedlegg 15
Høgskolens veileder: Endre Håland	Gradering: <Åpen>
Eventuelle Merknader: Vi tillater at oppgaven kan publiseres.	

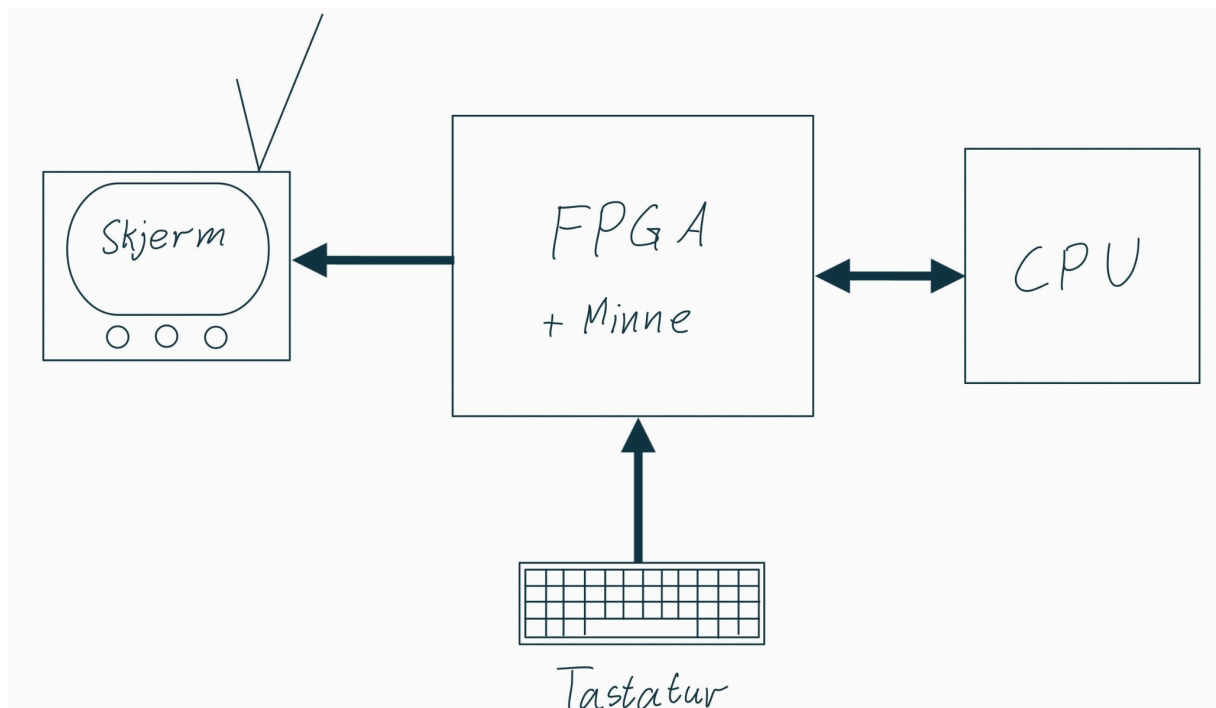
Oppdragsgiver: Selvvalgt	Oppdragsgivers referanse: ingen
Oppdragsgivers kontaktperson(er) (inkludert kontaktinformasjon): Sondre Johnsen og Jacob Kvasnes	

Revisjon	Dato	Status	Utført av
0.1	27.01.2026	Skrevet på innledning og kravspesifikasjon	Sondre og Jacob
0.2	03.02.2026	Skrevet viderer på kravspesifikasjon	Jacob
0.3	04.02.2026	Jobbet med analyse delen	Jacob
0.4	06.02.2026	Skrevet om deler av analyse delen	Jacob
0.5	07.02.2026	Skrevet på kravspesifikasjon og forklaring av CPU	Sondre
1	08.02.2026	Ferdig	Sondre og Jacob

Innhold

Dokumentkontroll	2
1 Innledning	4
1.1 Oppdragsgiver	4
1.2 Problemstilling	4
1.3 Hovedidé for løsningsforslag	4
2 Kravspesifikasjon	5
2.1 CPU	5
2.2 Hovedkort	6
2.3 Compiler	7
2.4 Programvare	8
2.4.1 «Wozmon»	8
2.4.2 «Pong»	8
2.4.3 «Snake»	8
2.4.4 «Tetris»	8
2.5 Dokumentasjon	8
3 Analyse av problemet	9
3.1 CPU	9
3.1.1 Arkitektur:	9
3.1.2 FPGA Prototype	9
3.1.3 PCB Prototyper	9
3.1.4 Endelige CPU Produkt:	10
3.2 Hovedkort	10
3.3 Software	10
3.3.1 Compiler	10
3.3.2 Programvare	11
3.3.3 Dokumentasjon	11
3.4 Vurderinger i forhold til verktøy og HW/SW komponenter	11
3.5 Konklusjon	11
3.5.1 CPU	11
3.5.2 Hovedkort	11
3.5.3 Software	11
3.5.4 Dokumentasjon	11

1 Innledning



1.1 Oppdragsgiver

Selvvalgt. Oppgaven stammer fra at Sondre Johnsen lagde en CPU, og Jacob Kvasnes ønsket å lage periferiutstyr og støttelogikk for CPU-en slik at man fikk en funksjonell datamaskin.

1.2 Problemstilling

Hovedformålet er å opparbeide erfaring med hvordan datamaskiner fungerer, ved å designe en egen 8-bit datamaskin. Resultatet skal være et produkt som andre kan bruke for å øke sin forståelse for datamaskinarkitektur.

1.3 Hovedidé for løsningsforslag

Hovedidéen er å lage en fungerende 8-bit datamaskin med en selvbygd 8-bit CPU som kobles til et FPGA-kort for å kommunisere med de forskjellige komponentene og periferiutstyr. En kompilator skal også lages så man enklere kan lage programvare for maskinen. Resterende tid av prosjektet vil bli brukt for å lage programvare og finpusse dokumentasjon.

2 Kravspesifikasjon

2.1 CPU

CPU Info	
Type	Value
Data Size	8Bit
Data Registers	4
Address Size	16bit
Memory Size	64kiB
Master Instructions	27
Sub Instructions	152
Logic Type	TTL

Operating Stats				
Type	Min	Typ	Max	Comment
Operating Voltage	4V	5V	12V	
Frequency	>0Hz	~	20kHz	
Power Draw	~1w	?	~20w	Not sure
Ambiant Temp	0 C*	~	40 C*	What we can test

Estimated Resources Used By Product CPU				
Type	Min	Est	Max	
Transistorer	1500	1850	2500	2N7002
Resistors	800	1000	1500	2.2k & 10k
LED	150	200	300	Red

Eget definert instruksjonsett:

Se vedlegg (Instruction_Set___Bachelor___HVL V0.2.pdf).

Pin assignment til FPGA:

8-bit data -inn/-ut

16-bit adresse

clk (klokke)

rst (reset)

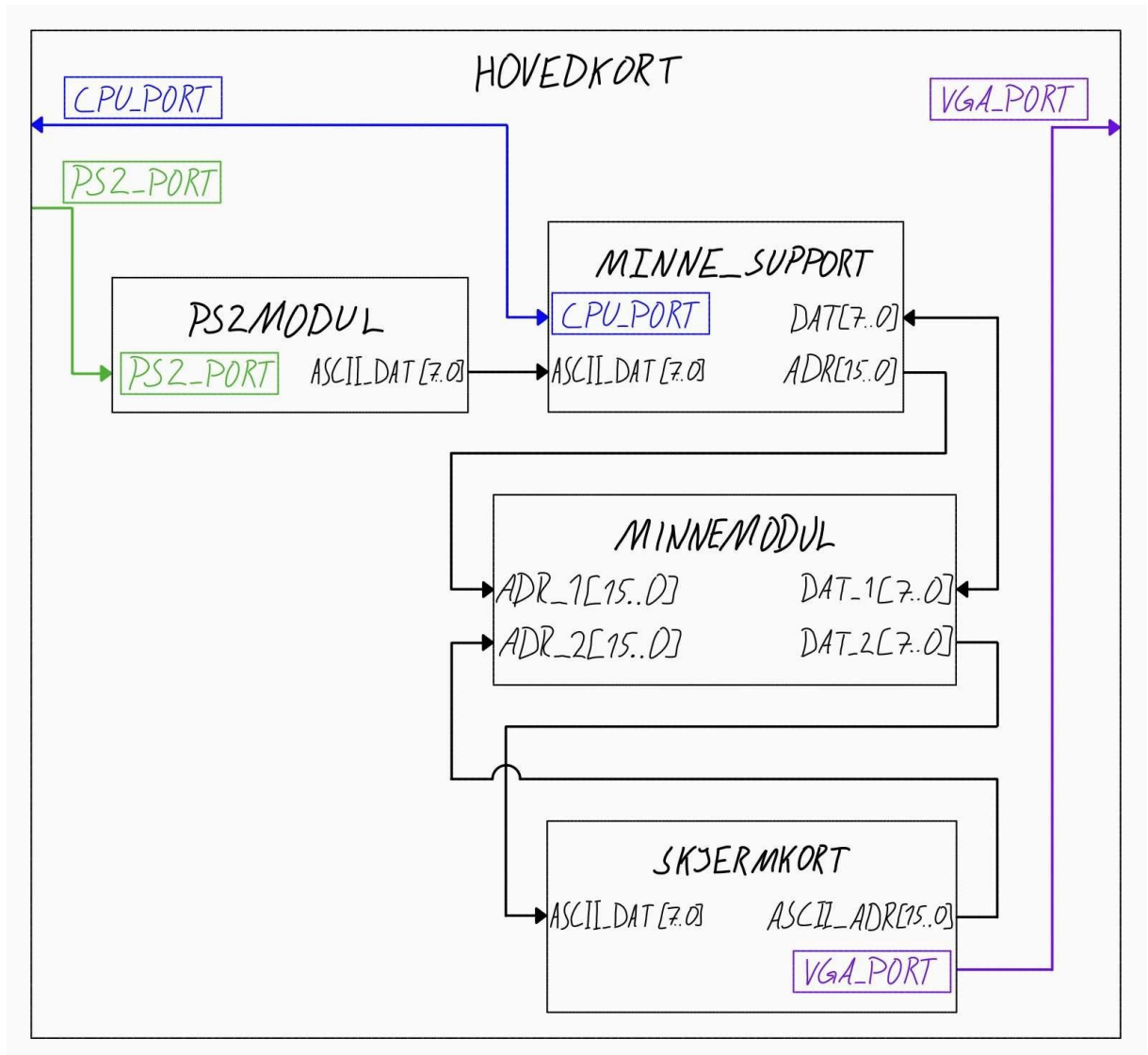
run (aktiver)

GND

5V

2.2 Hovedkort

Koble CPU-en til de andre komponentene slik at man får en fungerende datamaskin. Skal se noe lignende slik ut:



Kun datalinjer som består av mer enn én bit er tegnet for å holde skissen lettleselig.

“CPU_PORT”, “PS2_PORT” og “VGA_PORT” er signalene til/fra CPU, PS/2-tastaturet og VGA-skjerm.

De forskjellige modulen på kortet og deres krav:

- **PS2_MODUL**: Skal oversette dataen fra et PS/2-tastatur til ASCII-kode og sende det til MINNE_SUPPORT.

- **MINNE_SUPPORT:** Sin oppgave er å kommunisere mellom CPU, PS2_MODUL og MINNEMODUL.
- **MINNEMODUL:** 64kiB totalt minne med støtte for 50MHz klokkefrekvens og doble lese/skrivelinjer. Det vil allokeres 4.8kB med lagring i minne som vil bli brukt som VRAM (Video Random Access Memory) for skjermkortet.
- **SKJERMKORT:** Fra minneområdet tildelt i MINNEMODUL skal SKJERMKORT klare å sende et 640x480p 60Hz bilde over VGA. Formatet på dataen lagret i minneområdet skal være VGA-text-mode.

2.3 Compiler

Lage en kompilator for kode som ligner på C:

Stack og Heap:

Stack og eventuell Heap blir løst med programvare. Siden CPUen kommer ikke til å ha Stack-Pointer.

Variabler :

8-bit (char / int8_t)

16-bit (int16_t)

8-bit arrays

16-bit arrays

(for å gjøre det enkelt er alle variablene globale og derfor kan man ikke utføre vanlig rekursive funksjoner)

funksjoner:

Addisjon, subtraksjon, multiplikasjon, divisjon

metoder:

def

løkker:

for, while, foreach, dowhile

2.4 Programvare

Lage en terminal som har “Wozmon” funksjonalitet. Hvis det er tid, lages også “Pong”, “Snake” og “Tetris”.

2.4.1 «Wozmon»

Et program som vil funke som datamaskinen sin terminal og vil også ha samme funksjonalitet som det originale «Wozmon» programmet. Fra dette programmet skal man kunne starte andre program som ligger i minnet ved å skrive navnet på programmet.

2.4.2 «Pong»

To spillere kontrollerer vær sin “rektet” som har størrelse 1x6 på et brett som er 35x25 stort omringet av vegger. “Rektetene” ligger låst på x-aksen på hver sin side av brettet. “Rektetene” kan kontrolleres til å gå opp og ned. Det er en ping-pong ball på 2x2 størrelse. Ballen vil ha en konstant fart og bevege seg i en retning til den treffer en “rektet” eller topp-/bunn-veggen, treffer den en “rektet” vil ballen flippe sin x-fartsvektor og oppdatere y-fartsvektoren avhengig av hvor på rekteten ballen traff og treffer den en vegg flipper den y-fartsvektoren. Treffer ballen høyre-/venstre-vegg vil spilleren som kontrollerer “rekteten” lengst unna den gjeldende veggen få poeng. Det er første mann til 7 poeng.

2.4.3 «Snake»

Spilleren kontrollerer en slange med størrelse 3x1 på et 20x20 brett omringet av vegger. Spilleren tjener poeng ved å få slangen til å spise epler. Det er alltid et eple på brettet. Spiser slangen et eple, vil et nytt eple dukke opp på en tilfeldig plass og slangen sin lengde blir en mer. Spilleren taper hvis slangen kjører inn i seg selv eller veggene rundt brettet.

2.4.4 «Tetris»

Spilleren kontrollerer en og en fallende brikke på et 10x20 brett. Spilleren kan få brikken til å gjøre fire ting: 1. Gå raskere ned. 2. Gå til venstre. 3. Gå til høyre. 4. Roter brikken. Den fallende brikken er tilfeldig valgt fra én sekk med én av alle de mulige brikkene. Når sekken er tom for brikker fyller den seg opp igjen med én av alle de mulige brikkene i en tilfeldig rekkefølge. Når den kontrollerte brikken lander på en tidligere brikke eller «gulvet» til brettet, låser brikken seg fast og spilleren får en ny fallende brikke i midten av toppen på brettet. Hvis låste brikker dekker en hel rad slettes raden og spillerens poengsum økes. Hvis flere rader blir slettet samtidig vil spilleren få ekstra poeng. Spilleren taper hvis den nye brikken opprettes i en tidligere låst brikke.

2.5 Dokumentasjon

Alle moduler skal dokumenteres med datablad. Databladene skal inneholde timingdiagram, forklaring på alle innganger og utganger til modulen, og bruksveiledning.

passelige kondensatorer og LED (0402 pakke). Vi tenker å ha med mange LED lys som prøver å forklare hva som foregår i datamaskinen.

Det første av de fysiske versjonene blir å lage en versjon med splittete kretskort (et for hver del av CPUen) slik at eventuelle feilsøking blir mye lettere. Kan også kombinere FPGA CPUen og de individuelle eller flere av kretskortene for testing av kretskortene.

Blir det noen feil med den første fysiske versjonen skal de kretskortene oppdateres og bli bestilt på nytt.

3.1.4 Endelige CPU Produkt:

Til slutt skal det bestilles et stort kretskort som er kombinasjonen av de fungerende og oppdaterte kretskortene fra den første versjonen. Dette vil gjøre at tingene ser mer ryddig og fint, det vil også gjøre at man ikke trenger mange "Interconnects" mellom de separate kretskortene.

Selve "on-board" registerene på CPUen skal være separate kretskort som RAM sticks.

3.2 Hovedkort

De forskjellige modulene vil bli laget i rekkefølgen:

1. PS2MODUL
2. MINNEMODUL
3. SKJERMKORT
4. MINNE_SUPPORT

Tanken er at man kan bruke PS2MODUL i testing av MINNEMODUL som vil brukes i testing av SKJERMKORT. MINNE_SUPPORT blir sist da CPU-en er siste modul som kobles til hovedkortet.

Planlagt løsning for de forskjellige modulene:

- **PS2MODUL:** Lage først en komponent som får rådataen fra et PS/2-tastatur. Deretter lages en LUT (Look Up Table) for oversetting fra PS/2-rådata til ASCII.
- **MINNEMODUL:** Bruke en forhåndskonstruert minnemodul fra Quartus Prime Lite siden den har mulighet til å forhåndsinitialisere minnet med en .hex fil.
- **SKJERMKORT:** Lage en komponent som tar inn et xy-koordinat og pixel-data, som så sender ut VGA-PORT signalene. Deretter lage en komponent som kan lese fra VRAM-en for å se hvilken ASCII-karakter som skal tegnes, for så å bruke en LUT for å se hvilken pixel-data for angitt ASCII-karakter som skal sendes til den første modulen.
- **MINNE_SUPPORT:** Lage en komponent som passer på at ASCII-karakterer blir matet til CPU-en når den spør om det.

3.3 Software

3.3.1 Kompilator

Siden kompilatoren er en kompleks del av oppgaven, vil den bli jobbet med gjennom store deler av oppgaven.

3.3.2 Programvare

Siden tid kan bli et problem er den eneste programvaren som kan garanteres en terminal med "Wozmon" funksjonalitet. Hvis det er tid til å utvikle de andre programmene vil det bli gjort. Rekkefølgen på prioritering for programvaren er: "Wozmon", "Pong", "Snake", "Tetris".

3.3.3 Dokumentasjon

Dokumentasjon for modulene vil bli skrevet fortløpende gjennom prosjektet.

3.4 Vurderinger i forhold til verktøy og HW/SW komponenter

Hardware

Altera Cyclone IV E FPGA (EP4CE115) er FPGA brettet som brukes for utvikling av alle moduler.

Software:

Quartus Prime Lite 25.1 er programmet som blir brukt for FPGA utvikling og feilsøking

3.5 Konklusjon

3.5.1 CPU

Det vil bli laget flere versjoner av CPUen; en på FPGA og noen på Kretskort. Siden det skal lages en FPGA versjon først kan man teste systemet i helet før kretskortene er ferdig.

Digital logikk blir laget med TTL type.

Stort sett hele budsjettet vil bli allokert til fysisk produksjon / bestilling av kretskortene fra JLC

3.5.2 Hovedkort

Lage modulene i rekkefølgen:

1. PS2MODUL
2. MINNEMODUL - Bruker forhåndsdefinert modul fra Quartus
3. SKJERMKORT
4. MINNE_SUPPORT

3.5.3 Software

Kompilatoren vil Under utvikling av programvare vil terminalen med "Wozmon"-funksjoner bli prioritert. De andre programmene blir lagd hvis det er tid.

3.5.4 Dokumentasjon

Datablad og kretskorttegninger vil bli skrevet/produsert fortløpende gjennom hele prosjektet.

Forkortelser og ordforklaringer

OFDM	Orthogonal Frequency Division Multiplexing
CPU	Central Processing Unit
FPGA	Field Programmable Gate Array
VGA	Video Graphics Array
VRAM	Video Random Access Memory

Risikoliste

Risiko	Sannsynlighet	Konsekvens	Tiltak
Timing-problemer i FPGA designet	Høy	Ustabil system	Bruke simuleringsverktøy i Quartus for å unngå og løse timingproblemer.
CPU kretskort har feil i seg	Middels	Må fikse feil og bestille nytt PCBA kort. Så dyrere og tar mye tid	Bruke Simulerings programvare og teste fysisk kretsene på bread board før bestillingen og først bestille kretskortet i flere deler
Kompilator generer feil kode	Middels	Vanskelig feil søking.	Lager en enklere versjon av C.
Liten tid for programvare utvikling	Middels	Ikke alle program som er ønsket blir lagd	Eneste programvare som er lovet er en terminal med "Wozmon" funksjoner.
Jacob får problemer med Linux	Garantert	Stor tap av tid og potensielt mister man masse filer.	Jacob får ikke lov til å "distro-hoppe" og må jevnlig sikkerhetskopiere filene sine til Git.

Instruction Set - Simple Instruction Set (Not Completed)

You

February 10, 2026

1 Introduction

Explanation of the instruction set for the simple CPU (8-bit "homemade" CPU), each instruction will be explained how the interact with the various components of the cpu, the memory and other types of I/O.

Contents

1	Introduction	1
2	CPU FLAGS	3
2.1	Negative Flag	3
2.2	Zero Flag	3
2.3	Overflow Flag	4
2.4	Underflow Flag	4
2.5	Carry Flag	4
3	Register	5
4	Instructions	6
4.1	Jump	7
4.2	BOP - Branch On Positive	7
4.3	BON - Branch On Negative	7
4.4	BOZ - Branch On Zero	8
4.5	BNZ - Branch Not Zero	8
4.6	BOO - Branch On Overflow	8
4.7	BNO - Branch Not Overflow	9
4.8	BOU - Branch On Underflow	9
4.9	BNU - Branch Not Underflow	9
4.10	BOC - Branch On Carry	10
4.11	BNC - Branch Not Carry	10
4.12	RST - Reset	10
4.13	Clear	11
4.14	Load	11
4.15	Store	11
4.16	INC - Increment	12
4.17	DEC - Decriment	12
4.18	Right shift	12
4.19	Not	13
4.20	And	13
4.21	Or	14
4.22	Xor	14
4.23	Add - add without carry	15
4.24	Addc - add with carry	15
4.25	Move	16

2 CPU FLAGS

The CPU has 5 flags used for comparing values for conditional code jumping and code executions. Examples are If statement, Switch statements and For loops and While loops:

Character	Type
N	Negative
Z	Zero
O	Overflow
U	Underflow
C	Carry

2.1 Negative Flag

If the result of the last ALU-operation had a '1' as the MSB (Most Significant Bit) The Negative flag will become '1', as we are working with 2's complements.

Example:

Input 1	Input 2	Result
XXXXXXXXXX	XXXXXXXXXX	1XXXXXXXXX
XXXXXXXXXX	No input (00...)	1XXXXXXXXX

2.2 Zero Flag

If the result of the last ALU-operation had all bits equal to '0' then the Zero flag will become '1'.

Example:

Input 1	Input 2	Result
XXXXXXXXXX	XXXXXXXXXX	00000000
XXXXXXXXXX	No input (00...)	00000000

2.3 Overflow Flag

If the MSB in result is different than the registers going into the ALU, The Overflow flag will become '1'. AKA two **positive** numbers go in but the result is **negative**

Input 1	Input 2	Result
0XXXXXXXX	0XXXXXXXX	1XXXXXXXX
1XXXXXXXX	1XXXXXXXX	0XXXXXXXX
0XXXXXXXX	No input (00...)	1XXXXXXXX

2.4 Underflow Flag

Can only occur if the LSB (Least significant bit) is '1' and you **Right Shift** the value.

	Input 1	Result
Right Shift	XXXXXXXX1	0XXXXXXXX

2.5 Carry Flag

If in the add result of the last ALU-operation got too big and overflows above the 8'th and MSB bit the Carry Flag will be set

3 Register

There are 4 x 8-bit general purpose registers Register A, B, C and D. These registers can be used in the ALU and for loading to and from memory and be able to move between each other.

PC (program counter) is a 2x8 bit register that holds the memory address

4 Instructions

The CPU reads 8-bit Op-Codes and can only decode 26 different master instructions

nr	Assembly Instruction	OpCode
0	Jump	0x02
1	BOP	0x03
2	BON	0x04
3	BOZ	0x05
4	BNZ	0x06
5	BOO	0x07
6	BNO	0x08
7	BOU	0x09
8	BNU	0x0a
9	BOC	0x0b
10	BNC	0x0c
11	RST	0x0d
12-14	Clear	0x0e - 0x13
15	Load	0x14 - 0x17
16	Store	0x18 - 0x1b
17	Inc	0x1c - 0x1f
18	Dec	0x20 - 0x23
19	Right Shift	0x24 - 0x27
20	Not	0x28 - 0x2b
21	And	0x30 - 0x3f
22	Or	0x40 - 0x4f
23	Xor	0x50 - 0x5f
24	Add	0x60 - 0x6f
25	Addc	0x70 - 0x7f
26	Move	0x80 - 0x8f

4.1 Jump

Jump command is used to unconditionally jump to specific 16 bit (2 bytes) memory locations

Jump "00000010"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
Jump 0x1234	0x02	3	3	-	-	-	-	-	Setts PC to 0x1234

4.2 BOP - Branch On Positive

This Command Branches to a specific 16 bit (2 bytes) memory location only if flags : **Z** = '0' & **N** = '0'

BOP "00000011"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
BOP 0x1234	0x03	3	3	-	-	-	-	-	Setts PC to 0x1234 if N & Z = '0'

4.3 BON - Branch On Negative

Branches to a specific 16 bit (2 bytes) memory location only if flag : **N** = '1'

BON "00000100"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
BON 0x1234	0x04	3	3	-	-	-	-	-	Setts PC to 0x1234 if N = '1'

4.4 BOZ - Branch On Zero

Branches to a specific 16 bit (2 bytes) memory location
only if flag : Z = '1'

BOZ "00000101"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
BOZ 0x1234	0x05	3	3	-	-	-	-	-	Setts PC to 0x1234 if Z = '1'

4.5 BNZ - Branch Not Zero

Branches to a specific 16 bit (2 bytes) memory location
only if flag : Z = '0'

BNZ "00000110"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
BNZ 0x1234	0x06	3	3	-	-	-	-	-	Setts PC to 0x1234 if Z = '0'

4.6 BOO - Branch On Overflow

Branches to a specific 16 bit (2 bytes) memory location
only if flag : O = '1'

BOO "00000111"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
BOO 0x1234	0x07	3	3	-	-	-	-	-	Setts PC to 0x1234 if O = '1'

4.7 BNO - Branch Not Overflow

Branches to a specific 16 bit (2 bytes) memory location
only if flag : **O** = '0'

BNO "00001000"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
BNO 0x1234	0x08	3	3	-	-	-	-	-	Setts PC to 0x1234 if O = '0'

4.8 BOU - Branch On Underflow

Branches to a specific 16 bit (2 bytes) memory location
only if flag : **U** = '1'

BOU "00001001"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
BOU 0x1234	0x09	3	3	-	-	-	-	-	Setts PC to 0x1234 if U = '1'

4.9 BNU - Branch Not Underflow

Branches to a specific 16 bit (2 bytes) memory location
only if flag : **U** = '0'

BNU "00001010"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
BNU 0x1234	0x0a	3	3	-	-	-	-	-	Setts PC to 0x1234 if U = '0'

4.10 BOC - Branch On Carry

Branches to a specific 16 bit (2 bytes) memory location
only if flag : **C** = '1'

BOC "00001011"					Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C		
BOC 0x1234	0x0b	3	3	-	-	-	-	-	Setts PC to 0x1234 if C = '1'	

4.11 BNC - Branch Not Carry

Branches to a specific 16 bit (2 bytes) memory location
only if flag : **C** = '0'

BNC "00001100"					Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C		
BNC 0x1234	0x0C	3	3	-	-	-	-	-	Setts PC to 0x1234 if C = '0'	

4.12 RST - Reset

Resets everything about the cpu. (Dont use it; not synced)

RST "00001101"					Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C		
RST 0x1234	0x0d	1	1	-	-	-	-	-	Resets everything	

4.13 Clear

Clear can clear the flag Register, individual or all [registers](#)

Clear				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
Clear f	0x0e	1	1	0	0	0	0	0	Clears Flags
Clear r	0x0f	1	1	-	-	-	-	-	Clears all registers
Clear a	0x10	1	1	-	-	-	-	-	Clears register A
Clear b	0x11	1	1	-	-	-	-	-	Clears register B
Clear c	0x12	1	1	-	-	-	-	-	Clears register C
Clear d	0x13	1	1	-	-	-	-	-	Clears register D

4.14 Load

This command loads 1 byte from memory into one of the [registers](#)

Load "000101XX"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
Load a 0x1234	0x14	3	4	-	-	-	-	-	Loads from memory 0x1234 to register A
Load b 0x1234	0x15	3	4	-	-	-	-	-	Loads from memory 0x1234 to register B
Load c 0x1234	0x16	3	4	-	-	-	-	-	Loads from memory 0x1234 to register C
Load d 0x1234	0x17	3	4	-	-	-	-	-	Loads from memory 0x1234 to register D

4.15 Store

This command stores / saves the select [registers](#) into memory

Store "000110XX"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
Store a 0x1234	0x18	3	4	-	-	-	-	-	Stores Register A to memory 0x1234
Store b 0x1234	0x19	3	4	-	-	-	-	-	Stores Register B to memory 0x1234
Store c 0x1234	0x1a	3	4	-	-	-	-	-	Stores Register C to memory 0x1234
Store d 0x1234	0x1b	3	4	-	-	-	-	-	Stores Register D to memory 0x1234

4.16 INC - Increment

This command increments one of the [registers](#)

Inc "000111XX"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
Inc a	0x1c	1	1	+	+	+	0	+	Increments register A
Inc b	0x1d	1	1	+	+	+	0	+	Increments register B
Inc c	0x1e	1	1	+	+	+	0	+	Increments register C
Inc d	0x1f	1	1	+	+	+	0	+	Increments register D

4.17 DEC - Decriment

This command decrements one of the [registers](#)

Dec "001000XX"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
Dec a	0x20	1	1	+	+	+	0	+	Decriments register A
Dec b	0x21	1	1	+	+	+	0	+	Decriments register B
Dec c	0x22	1	1	+	+	+	0	+	Decriments register C
Dec d	0x23	1	1	+	+	+	0	+	Decriments register D

4.18 Right shift

This command right shifts the selected [registers](#)

Rs "001001XX"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
Rs a	0x24	1	1	0	+	+	+	0	Right Shifts A
Rs b	0x25	1	1	0	+	+	+	0	Right Shifts B
Rs c	0x26	1	1	0	+	+	+	0	Right Shifts C
Rs d	0x27	1	1	0	+	+	+	0	Right Shifts D

4.19 Not

This command not one of the [registers](#)

Not "001010XX"				Set Flags					Comment
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
Not A	0x28	1	1	+	+	+	0	0	Nots Register A
Not B	0x29	1	1	+	+	+	0	0	Nots Register B
Not C	0x2a	1	1	+	+	+	0	0	Nots Register C
Not D	0x2b	1	1	+	+	+	0	0	Nots Register D

4.20 And

This command and's 2 [registers](#)

And "0011XXXX"				Set Flags					Comment (Registers)
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
And A A	0x30	1	1	+	+	0	0	0	A <= A and A
And B A	0x31	1	1	+	+	0	0	0	B <= B and A
And C A	0x32	1	1	+	+	0	0	0	C <= C and A
And D A	0x33	1	1	+	+	0	0	0	D <= D and A
And A B	0x34	1	1	+	+	0	0	0	A <= A and B
And B B	0x35	1	1	+	+	0	0	0	B <= B and B
And C B	0x36	1	1	+	+	0	0	0	C <= C and B
And D B	0x37	1	1	+	+	0	0	0	D <= D and B
And A C	0x38	1	1	+	+	0	0	0	A <= A and C
And B C	0x39	1	1	+	+	0	0	0	B <= B and C
And C C	0x3a	1	1	+	+	0	0	0	C <= C and C
And D C	0x3b	1	1	+	+	0	0	0	D <= D and C
And A D	0x3c	1	1	+	+	0	0	0	A <= A and D
And B D	0x3d	1	1	+	+	0	0	0	B <= B and D
And C D	0x3e	1	1	+	+	0	0	0	C <= C and D
And D D	0x3f	1	1	+	+	0	0	0	D <= D and D

4.21 Or

This command or's 2 [registers](#).

Or "0100XXXX"				Set Flags					Comment (Registers)
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
Or A A	0x40	1	1	+	+	0	0	0	A <= A or A
Or A B	0x41	1	1	+	+	0	0	0	B <= B or A
Or A C	0x42	1	1	+	+	0	0	0	C <= C or A
Or A D	0x43	1	1	+	+	0	0	0	D <= D or A
Or B A	0x44	1	1	+	+	0	0	0	A <= A or B
Or B B	0x45	1	1	+	+	0	0	0	B <= B or B
Or B C	0x46	1	1	+	+	0	0	0	C <= C or B
Or B D	0x47	1	1	+	+	0	0	0	D <= D or B
Or C A	0x48	1	1	+	+	0	0	0	A <= A or C
Or C B	0x49	1	1	+	+	0	0	0	B <= B or C
Or C C	0x4a	1	1	+	+	0	0	0	C <= C or C
Or C D	0x4b	1	1	+	+	0	0	0	D <= D or C
Or D A	0x4c	1	1	+	+	0	0	0	A <= A or D
Or D B	0x4d	1	1	+	+	0	0	0	B <= B or D
Or D C	0x4e	1	1	+	+	0	0	0	C <= C or D
Or D D	0x4f	1	1	+	+	0	0	0	D <= D or D

4.22 Xor

This command xor's 2 [registers](#)

Xor "0101XXXX"				Set Flags					Comment (Registers)
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
Xor A A	0x50	1	1	+	+	+	0	0	A <= A xor A
Xor A B	0x51	1	1	+	+	+	0	0	B <= B xor A
Xor A C	0x52	1	1	+	+	+	0	0	C <= C xor A
Xor A D	0x53	1	1	+	+	+	0	0	D <= D xor A
Xor B A	0x54	1	1	+	+	+	0	0	A <= A xor B
Xor B B	0x55	1	1	+	+	+	0	0	B <= B xor B
Xor B C	0x56	1	1	+	+	+	0	0	C <= C xor B
Xor B D	0x57	1	1	+	+	+	0	0	D <= D xor B
Xor C A	0x58	1	1	+	+	+	0	0	A <= A xor C
Xor C B	0x59	1	1	+	+	+	0	0	B <= B xor C
Xor C C	0x5a	1	1	+	+	+	0	0	C <= C xor C
Xor C D	0x5b	1	1	+	+	+	0	0	D <= D xor C
Xor D A	0x5c	1	1	+	+	+	0	0	A <= A xor D
Xor D B	0x5d	1	1	+	+	+	0	0	B <= B xor D
Xor D C	0x5e	1	1	+	+	+	0	0	C <= C xor D
Xor D D	0x5f	1	1	+	+	+	0	0	D <= D xor D

4.23 Add - add without carry

This command adds 2 [registers](#)

Add "0110XXXX"				Set Flags					Comment (Registers)
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
Add A A	0x60	1	1	+	+	+	0	+	A <= A + A
Add A B	0x61	1	1	+	+	+	0	+	B <= B + A
Add A C	0x62	1	1	+	+	+	0	+	C <= C + A
Add A D	0x63	1	1	+	+	+	0	+	D <= D + A
Add B A	0x64	1	1	+	+	+	0	+	A <= A + B
Add B B	0x65	1	1	+	+	+	0	+	B <= B + B
Add B C	0x66	1	1	+	+	+	0	+	C <= C + B
Add B D	0x67	1	1	+	+	+	0	+	D <= D + B
Add C A	0x68	1	1	+	+	+	0	+	A <= A + C
Add C B	0x69	1	1	+	+	+	0	+	B <= B + C
Add C C	0x6a	1	1	+	+	+	0	+	C <= C + C
Add C D	0x6b	1	1	+	+	+	0	+	D <= D + C
Add D A	0x6c	1	1	+	+	+	0	+	A <= A + D
Add D B	0x6d	1	1	+	+	+	0	+	B <= B + D
Add D C	0x6e	1	1	+	+	+	0	+	C <= C + D
Add D D	0x6f	1	1	+	+	+	0	+	D <= D + D

4.24 Addc - add with carry

This command adds 2 [registers](#) + [Carry](#)

Addc "0111XXXX"				Set Flags					Comment (Registers)
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
Addc A A	0x70	1	1	+	+	+	0	+	A <= A + A + Carry
Addc A B	0x71	1	1	+	+	+	0	+	B <= B + A + Carry
Addc A C	0x72	1	1	+	+	+	0	+	C <= C + A + Carry
Addc A D	0x73	1	1	+	+	+	0	+	D <= D + A + Carry
Addc B A	0x74	1	1	+	+	+	0	+	A <= A + B + Carry
Addc B B	0x75	1	1	+	+	+	0	+	B <= B + B + Carry
Addc B C	0x76	1	1	+	+	+	0	+	C <= C + B + Carry
Addc B D	0x77	1	1	+	+	+	0	+	D <= D + B + Carry
Addc C A	0x78	1	1	+	+	+	0	+	A <= A + C + Carry
Addc C B	0x79	1	1	+	+	+	0	+	B <= B + C + Carry
Addc C C	0x7a	1	1	+	+	+	0	+	C <= C + C + Carry
Addc C D	0x7b	1	1	+	+	+	0	+	D <= D + C + Carry
Addc D A	0x7c	1	1	+	+	+	0	+	A <= A + D + Carry
Addc D B	0x7d	1	1	+	+	+	0	+	B <= B + D + Carry
Addc D C	0x7e	1	1	+	+	+	0	+	C <= C + D + Carry
Addc D D	0x7f	1	1	+	+	+	0	+	D <= D + D + Carry

4.25 Move

This command moves the value of one [register](#) into another register

Move "1000XXXX"				Set Flags					Comment (Registers)
Ex Instruction	OpCode	Bytes	Clocks	N	Z	O	U	C	
Move A A	0x80	1	1	-	-	-	-	-	A <= A
Move A B	0x81	1	1	-	-	-	-	-	B <= A
Move A C	0x82	1	1	-	-	-	-	-	C <= A
Move A D	0x83	1	1	-	-	-	-	-	D <= A
Move B A	0x84	1	1	-	-	-	-	-	A <= B
Move B B	0x85	1	1	-	-	-	-	-	B <= B
Move B C	0x86	1	1	-	-	-	-	-	C <= B
Move B D	0x87	1	1	-	-	-	-	-	D <= B
Move C A	0x88	1	1	-	-	-	-	-	A <= C
Move C B	0x89	1	1	-	-	-	-	-	B <= C
Move C C	0x8a	1	1	-	-	-	-	-	C <= C
Move C D	0x8b	1	1	-	-	-	-	-	D <= C
Move D A	0x8c	1	1	-	-	-	-	-	A <= D
Move D B	0x8d	1	1	-	-	-	-	-	B <= D
Move D C	0x8e	1	1	-	-	-	-	-	C <= D
Move D D	0x8f	1	1	-	-	-	-	-	D <= D

References